

Module 8

Topics: Using arrays, Naming of variables in datasets

Files to download: mod8_A.sas7bdat, mod8_B.sas7bdat, mod8_C.sas7bdat, mod8_D.sas7bdat, pets.sas7bdat

8.1 - Using arrays and naming of variables in SAS datasets

In DATA step programming you can put variables into a temporary group called an array. You might want to do this when you would like to combine the values of certain variables in order to create a new variable.

Array Basics

The basic format of an array looks something like this:

Array array-name(number-of-elements) array-elements;

Where **array-name** identifies the group of variables in the array (a SAS name that you choose to identify the group of variables that make up the array), **number-of-elements** identifies the number of variables in the array (enclosed in braces, tells the SAS system how many variables you are grouping), and **array-elements** are the elements or fields that make up the array (variable-1 to variable-n lists the variable names).

Array Tips

- The array-name can't be the same as the name of a SAS variable used in that data step.
- Number-of-elements can be a number, a calculation, a numeric variable, or an asterisk. An asterisk is used when the number of elements is unknown and you need SAS to count them (this is always enclosed in brackets or parentheses).
- Arrays can be character or numeric, but not both.
- Array-name and array-elements must be valid SAS names (between 1-32 characters, beginning with a letter or an underscore, no blanks, no reserved SAS words. etc.).

Array Example 1

In this example, we have a dataset (mod8_A.sas7bdat) with columns for studyid and 15 variables (var1 – var15) that contain values of 0, 1, or 9, where 0=No, 1=Yes, and 9=Missing. Let's say we want to convert all the missing values (9's) to missing (.).

AEPI537D

With regular IF statements, we would have to do something like this:

```
LIBNAME NICOLE 'H:\AEPI537D';

data METHOD1;
    set NICOLE.mod8_A;
        if var1=9 then var1=.;
        if var2=9 then var2=.;
        if var3=9 then var3=.;
        if var4=9 then var4=.;
        if var5=9 then var5=.;
        if var6=9 then var6=.;
        if var7=9 then var7=.;
        if var8=9 then var8=.;
        if var9=9 then var9=.;
        if var10=9 then var10=.;
        if var11=9 then var11=.;
        if var12=9 then var12=.;
        if var13=9 then var13=.;
        if var14=9 then var14=.;
        if var15=9 then var15=.;
run;
```

But with an array, we could do something like this:

```
data METHOD2;
    set NICOLE.mod8_A;
    array blanks {15} var1 var2 var3 var4 var5 var6 var7 var8 var9 var10 var11
                    var12 var13 var14 var15;
    do i = 1 to 15;
        if blanks {i} = 9 then blanks {i} = . ;
    end;
run;
```

So with method 2, we created an array called “blanks” and specified 15 variables (var1-var15). We then asked SAS to loop through these 15 variables (do i = 1 to 15) and perform the step (if the value is 9 make the value .).

Array Example 2

In this example, we want to create a new indicator variable called SCORE that is the sum of the values of the variables TEST1, TEST2, and TEST3. A very simple way to do this would be to simply add the values using the following program.

```
LIBNAME NICOLE 'H:\AEPI537D';

DATA TEMP1;
    SET NICOLE.mod8_B;
        SCORE = TEST1 + TEST2 + TEST3;
RUN;
```

```
PROC FREQ DATA = TEMP1;
    TABLES SCORE;
RUN;
```

To demonstrate the use of an array we will perform the same operation using an array.

```
DATA TEMP2;
    SET TEMP1;
    SCORE2 = 0; *This is a second version of SCORE that we will calculate using an array;

    ARRAY SCARRAY {3} TEST1 TEST2 TEST3;

    DO i = 1 TO 3;
        IF SCARRAY {i} = 1 THEN SCORE2 = SCORE2 + 1;
        IF SCARRAY {i} = 0 THEN SCORE2 = SCORE2 + 0;
        IF SCARRAY {i} = . THEN SCORE2 = .;
    END;
RUN;
```

Notice that you still need to take care of the missing values when using an array. Let's take a look at a cross-tab of these two variables to make sure we got the same results.

```
PROC FREQ DATA = TEMP2;
    TABLES SCORE*SCORE2;
RUN;
```

In this example, using an array was more complicated than the program that simply added the three variables together. However, there are situations that make the use of an array much more efficient than any other technique.

Array Example 3

The SAS dataset mod8_C has a variable that stores information about a history of blood clots among relatives called RELTCLOT that is coded 1 if yes, and 0 if no. There are also 10 variables indicating the location(s) of the blood clot, LOCATN1 through LOCATN10. These location variables have the following values: 1= veins under the skin, 2= deep veins of the legs, 3= arterial thrombosis, 4= heart, 5= brain, 6= lungs, 7= eye, 8= other, and 9= don't know. For each individual that has a family history (RELTCLOT=1) they could have anywhere from 1 to 10 of the location variables filled in with one of those values 1 through 9.

Let's say that we want to create a variable that specifies whether the subject has a family history of venous blood clots, yes or no. In order to qualify as a venous blood clot, the location of the clot must be in one of the following locations: the veins under the skin, the deep veins of the legs, the lungs, or the eye. See above for the values that would need to be entered into the 10 variables for location named LOCATN1 through LOCATN10.

The following program will use an array to create a family history of venous clots variable that uses data about the presence and location of any blood clots among family members.

AEPI537D

```
DATA TEMP1;
SET NICOLE.mod8_C;

LOC = 0;

ARRAY HISTCLOT{10} LOCATN1 LOCATN2 LOCATN3 LOCATN4 LOCATN5 LOCATN6 LOCATN7
LOCATN8 LOCATN9 LOCATN10;

DO i = 1 TO 10;
IF HISTCLOT{i} IN (1,2,6,7) THEN LOC = 1;
END;

IF RELTCLLOT = 1 AND LOC = 1 THEN FAMHIS = 1;
ELSE FAMHIS = 0;

RUN;

PROC FREQ DATA = TEMP1;
TABLES RELTCLLOT*LOC*FAMHIS / LIST;
RUN;
```

In this example, you have used a simple array to create a single yes/no variable for family history of blood clots from a long list of variables. Without the array, the programming to create this variable would be very long and tedious.

Array Example 4

The next example illustrates the use of an array to create a variable that sums drug adherence. In the mod8_D dataset, there are several drug variables. The values for each drug variable are: 1= 7 days/wk, 2= 6 days/wk, 3= 5 days/wk, 4= 4 days/wk, 5= 3 days/wk, 6= 2 days/wk or less, 8= don't know, 9= refuse. We are interested in the total number of drugs taken 6 or 7 days a week, so we are looking to count the drug if the value is a 1 (7 days/wk) or a 2 (6 days/wk).

```
DATA TEMP3;
SET NICOLE.mod8_D;

DRUGS = 0;

ARRAY DRGARRAY {21} DRUG1A DRUG1B DRUG1C DRUG2A DRUG2B DRUG2C
DRUG3A DRUG3B DRUG3C DRUG4A DRUG4B DRUG4C
DRUG5A DRUG5B DRUG5C DRUG6A DRUG6B DRUG6C
DRUG7A DRUG7B DRUG7C;

DO j=1 TO 21;
IF DRGARRAY {j} = 1 OR DRGARRAY {j} = 2 THEN DRUGS = DRUGS + 1;
IF DRGARRAY {j} >=3 THEN DRUGS = DRUGS + 0;
END;

RUN;

PROC FREQ DATA=TEMP3;
TABLES DRUGS;
RUN;
```

To check to make sure that your program correctly calculated the total number of drugs, you would want to check several study subjects in detail. You could do this in the following way.

```
PROC PRINT DATA=TEMP3;
  VAR DRUG1A DRUG1B DRUG1C DRUG2A DRUG2B DRUG2C DRUG3A DRUG3B DRUG3C DRUG4A
      DRUG4B DRUG4C DRUG5A DRUG5B DRUG5C DRUG6A DRUG6B DRUG6C DRUG7A DRUG7B
      DRUG7C DRUGS;
  WHERE ID = 4;
RUN;
```

You have now created a single variable called DRUGS which added up the number of drugs (with appropriate adherence) for each study subject. If you had not used an array, you would have had to write a very long program with many if-then statements to create this variable.

8.2 - Bonus Info Related to Addition and Missing Values in SAS

Earlier in this module we created a variable called SCORE by adding the values of TEST1, TEST2, and TEST3.

```
DATA TEMP1;
  SET NICOLE.mod8_B;
  SCORE = TEST1 + TEST2 + TEST3;
RUN;
```

This was pretty straight forward, but it is worth noting that there is more than one way to do addition with variables in SAS, and the method you choose depends on whether you have missing values (and how you would like to treat those missing values).

Let's make a quick dataset to demonstrate. The dataset ADDITION has variables called var1, var2, and var3, each with values of 0, 1, or missing.

```
data addition;
input studyid var1 var2 var3;
datalines;
1001 1 1 1
1002 1 0 1
1003 1 1 1
1004 0 0 0
1005 . 1 1
1006 1 0 1
1007 0 0 .
1008 1 . 0
1009 . . 1
1010 1 0 1
;
run;
```

We will use the following code to demonstrate how SAS handles missing values with 2 different methods of addition.

```
data addition_rev;
  set addition;

  /*method 1*/ sum_w_plus = var1 + var2 + var3;

  /*method 2*/ sum_w_sum = sum(var1 , var2 , var3);

run;
```

In method 1 we are creating a variable called sum_w_plus where we use simple plus signs to add the values of var1, var2, and var3. In method 2 we are creating a variable called sum_w_sum where we use the SUM function to add the values of var1, var2, and var3.

Note the difference in the output below. Method 1 (regular addition) produced a missing value for the new variable if any of the variables in the equation had a missing value. Method 2 (sum function) added whatever values were there and provided a total in the new variable, ignoring any missing values.

VIEWTABLE: Work.Addition_rev						
	studyid	var1	var2	var3	sum_w_plus	sum_w_sum
1	1001	1	1	1	3	3
2	1002	1	0	1	2	2
3	1003	1	1	1	3	3
4	1004	0	0	0	0	0
5	1005	.	1	1	.	2
6	1006	1	0	1	2	2
7	1007	0	0	.	.	0
8	1008	1	.	0	.	1
9	1009	.	.	1	.	1
10	1010	1	0	1	2	2

Typically you will probably want to take control of your own data and determine what happens with your missing values before you start creating new variables and doing calculations. However, it is important to understand how different SAS procedures will handle any missing values. Sometimes it won't matter to you at all if some of the items were missing and you will want the score for whatever is present. Other times, it could be important to avoid calculating a total if there are any missing values. For example, if I have a depression scale in my survey with 10 items and one participant answers 5 questions and another answers all 10 but they both have a score of 5, do those scores mean the same thing? Probably not. It would be important in that situation to write if/then statements to limit those receiving a score OR being familiar with the different methods of addition to know how those will be calculated.

MOD 8: USE SAS COMMENTS TO NUMBER YOUR WORK.

1. What is the mistake in the following array?

```
DATA NEW;
  SET ORIG;
  ARRAY BLANKS {4} VAR1 VAR2 VAR3 VAR4 VAR5 VAR6;
  DO i = 1 to 6;
    IF BLANKS {i} = 9 THEN BLANKS {i} = . ;
  END;
RUN;
```

2. The dataset PETS.sas7bdat has the following variables indicating which pets people own. Those that said Yes to the binary Other variable could fill in up to 4 text variables with additional pets.

```
Dog (1=yes, 0=no)
Cat (1=yes, 0=no)
Bird (1=yes, 0=no)
Fish (1=yes, 0=no)
Other (1=yes, 0=no)
Other_spec1 (text field filled in only when Other=1)
Other_spec2 (text field filled in only when Other=1)
Other_spec3 (text field filled in only when Other=1)
Other_spec4 (text field filled in only when Other=1)
```

Create an array which counts the number of pet types in each home (dog, cat, bird, fish). Call this new variable PETNUM. Do not include the Other pet variable at this point. Call this dataset TEMP1. Run a PROC FREQ on PETNUM.

3. Use PROC PRINT take a look at the ID variable and Other_spec1, Other_spec2, Other_spec3, Other_spec4 for only people who answered “yes” Other so that you can see what these “other” pets look like.
4. Create an array to count the number of “other” write-in pets. Call this dataset TEMP2 and create it from TEMP1. This variable will be named OTHRPETS. Hint: Have the array count it if it is not missing (i.e., there is any pet listed in the other write-in field) and not count it if it is missing.
5. Write a program that calculates the total number of pets for each home. Store this total in a variable called ALLPETS. Hint: You don’t need to do an array for this. You already have 1 variable for total number of dog, cat, bird, and fish (PETNUM) and 1 variable for the other write-ins (OTHRPETS), so just combine these. Run a procedure to check that the value of ALLPETS was calculated correctly.